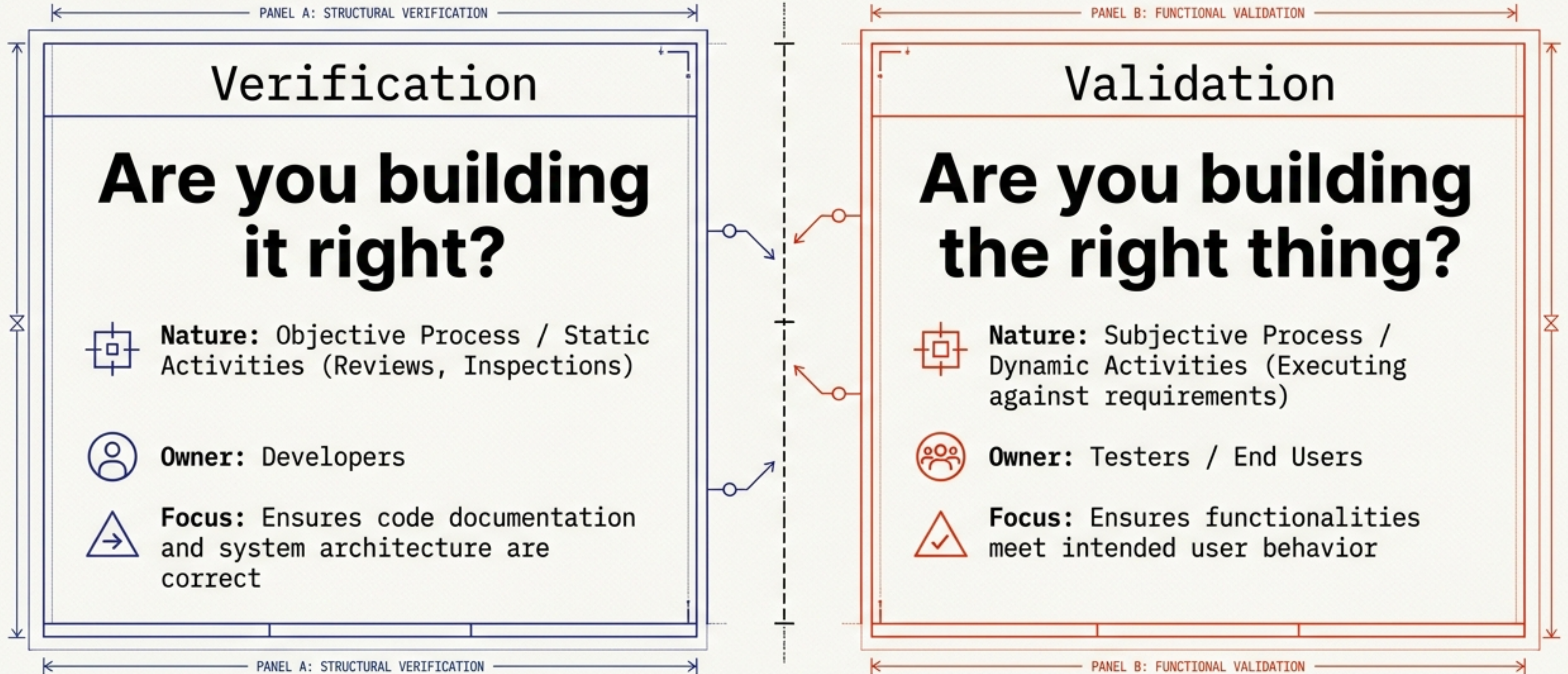
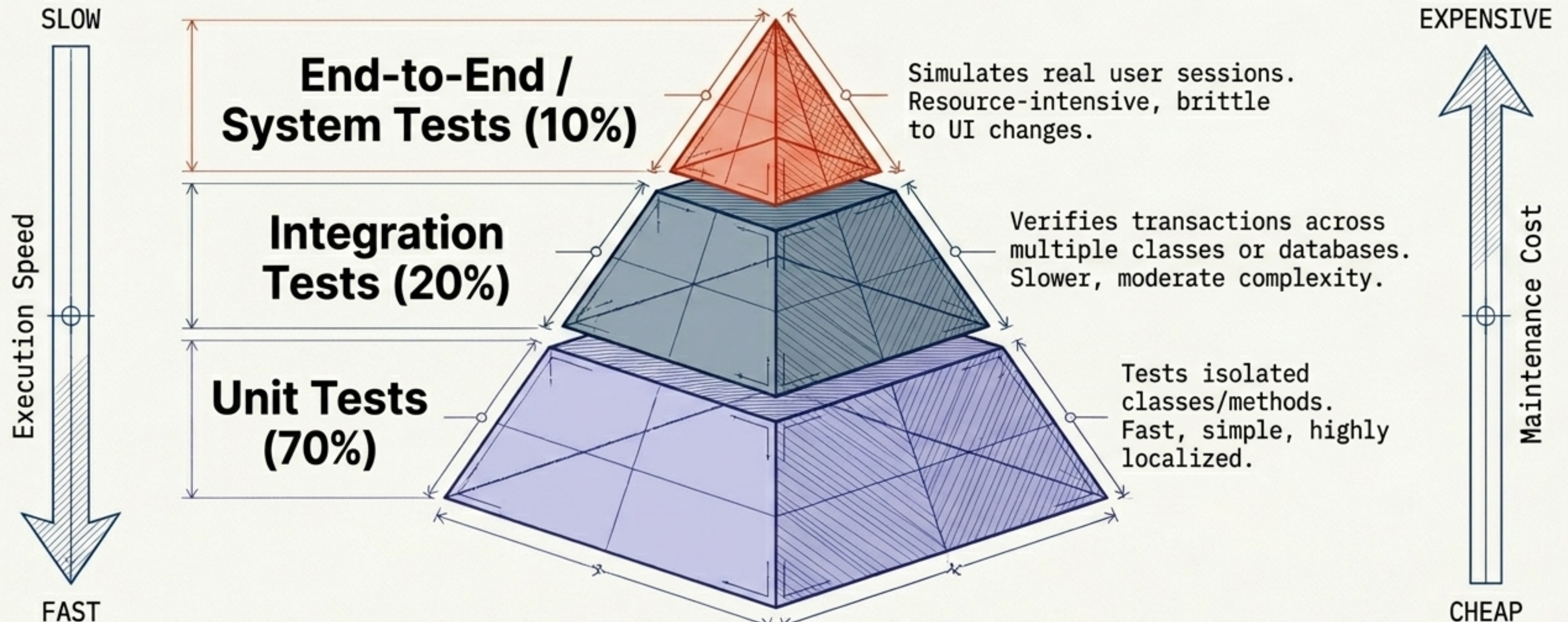


The Philosophy of Testing



The Testing Pyramid: A Strategic Blueprint

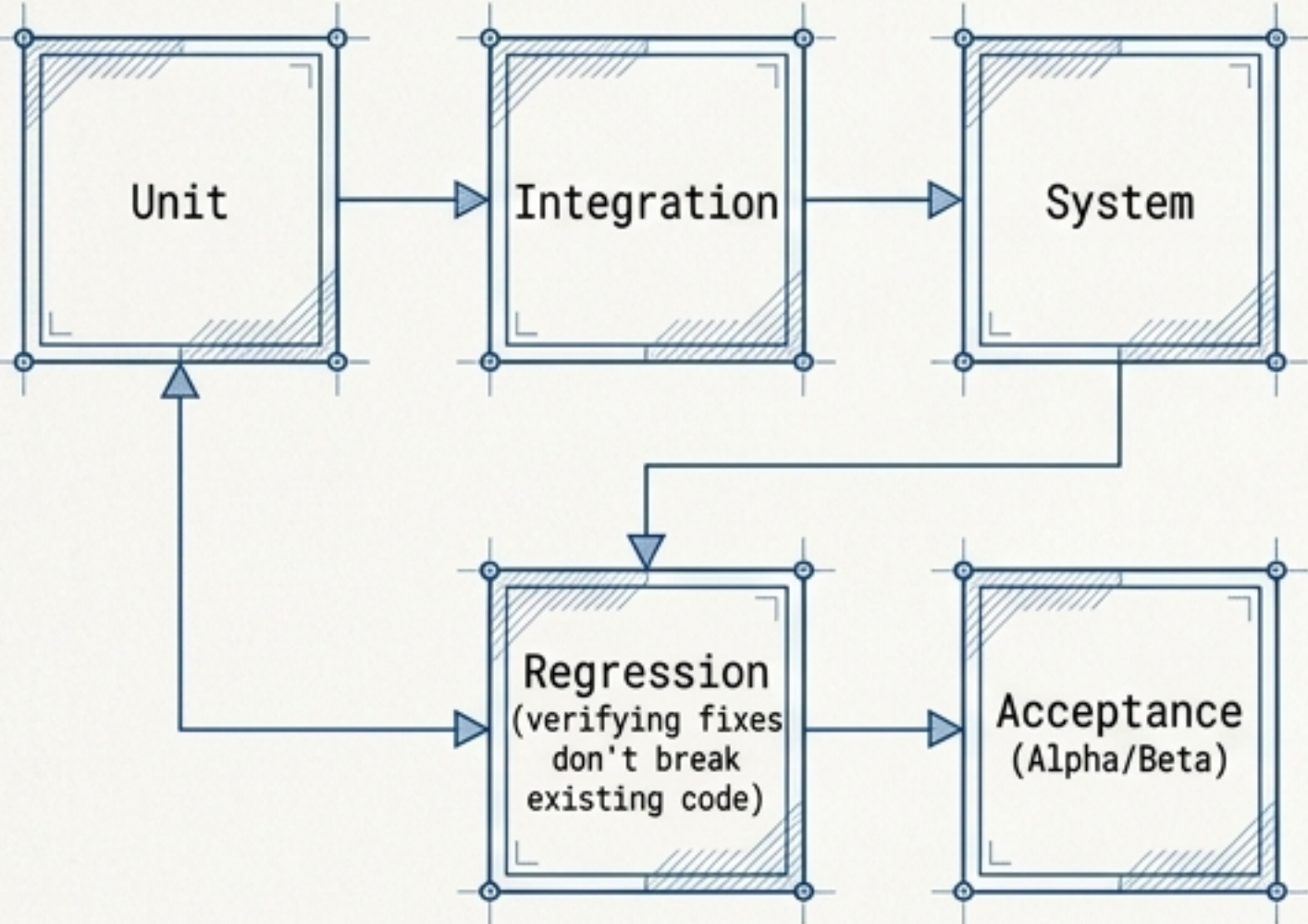


Recommendation: 70% Unit, 20% Integration, 10% End-to-end.

Categorizing the Testing Landscape

Functional Testing

Tests: What the system does.



Non-Functional Testing

Tests: How well the system does it.



Performance
(bottlenecks)



Load
(max capacity)



Stress
(beyond breaking points)



Usability
(user observation)



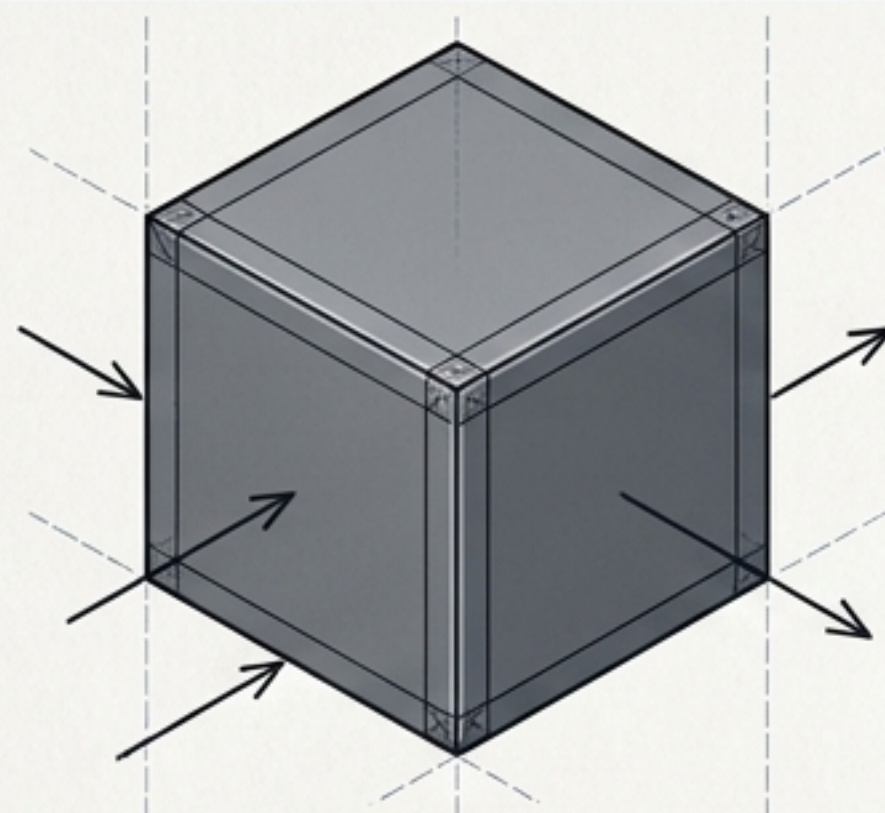
Sectrility
(user observation)



Security &
Portability

The Methodology Spectrum

Black-Box (Functional)

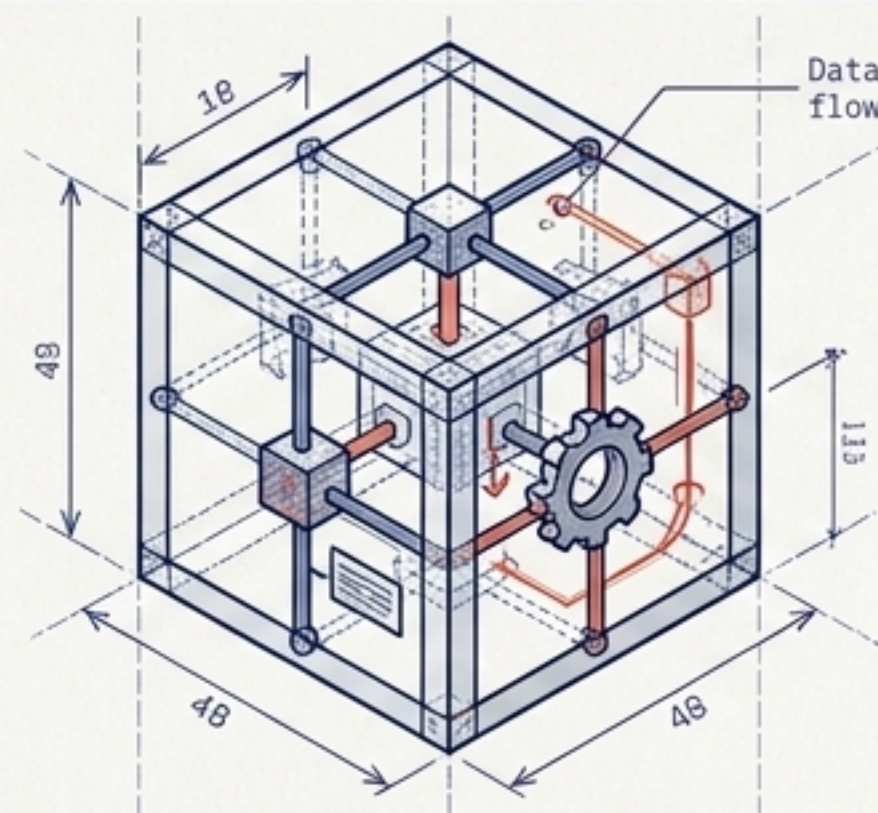


Knowledge Level:
Oblivious to architecture;
relies on inputs/outputs.

Target Focus:
Large code segments, UI, external
expectations.

Execution:
Least time-consuming, blind coverage.

White-Box (Structural)

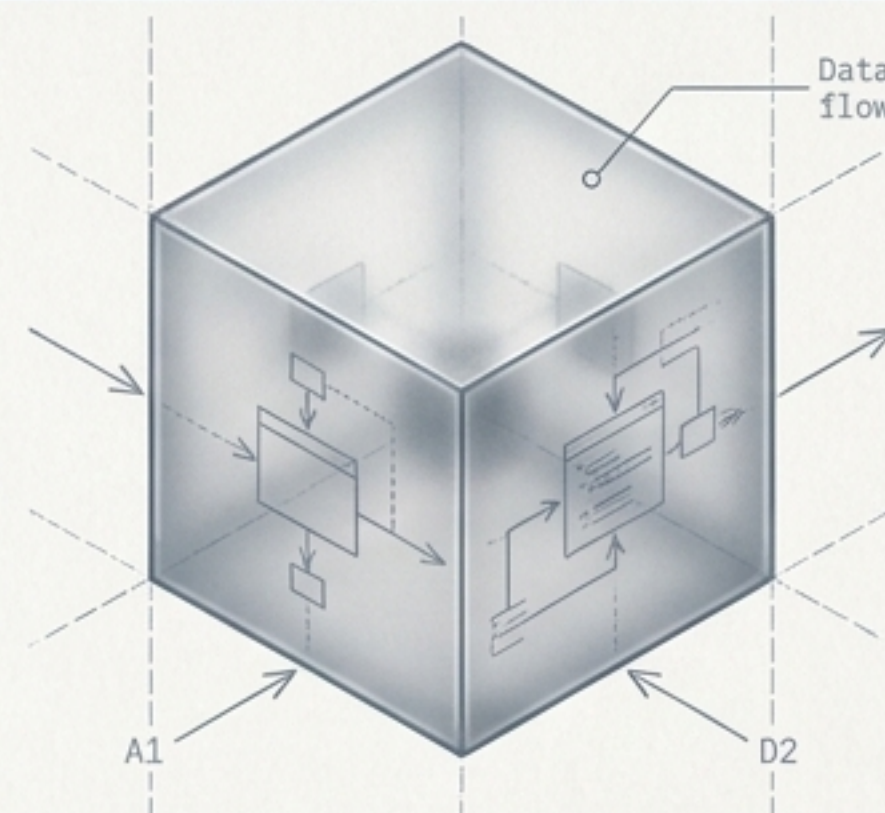


Knowledge Level:
Full access to source code and
internal logic.

Target Focus:
Specific code segments, internal
boundaries, algorithms.

Execution:
Most exhaustive and time-consuming.

Grey-Box (Translucent)

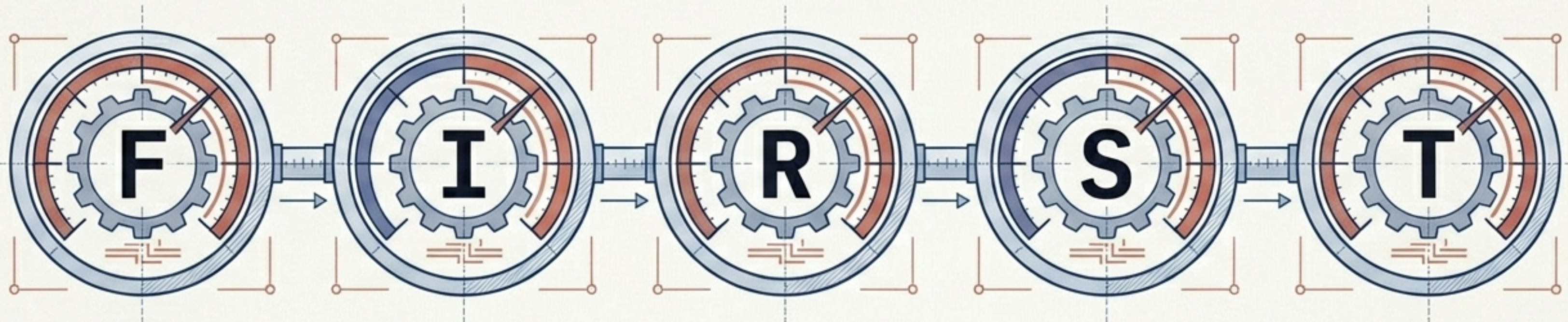


Knowledge Level:
Relies on interface definitions
and data flow diagrams.

Target Focus:
Communication protocols, data type
handling.

Execution:
Balanced efficiency.

The Blueprint of a Good Test



Fast

Runs in milliseconds.
Isolate slow tests.

Independent

Order of execution
does not matter.
No shared states.

Repeatable

Deterministic.
Eradicates flaky
tests.

Self-checking

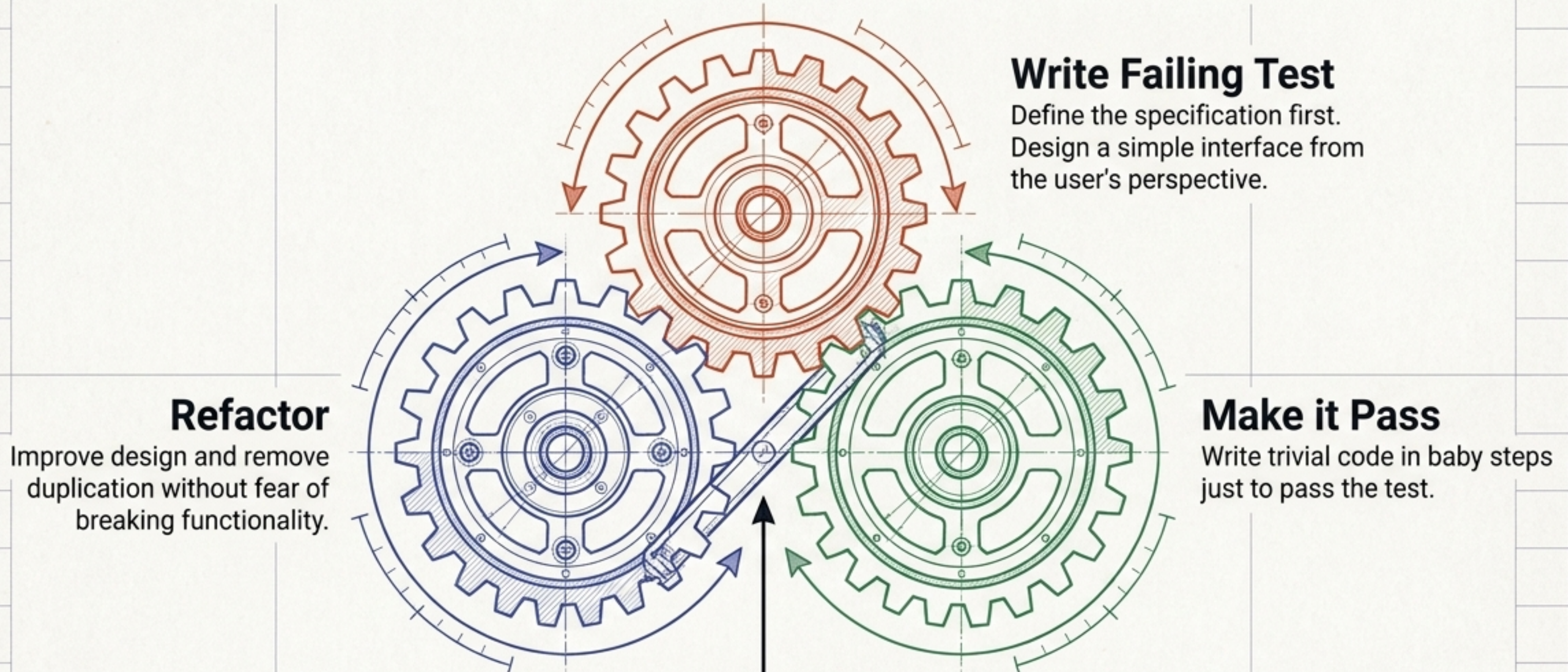
IDE output is binary:
Pass or Fail.
No manual parsing.

Timely

Written before
production code.

Code without tests is bad code. – Michael Feathers

Test-Driven Development (TDD)



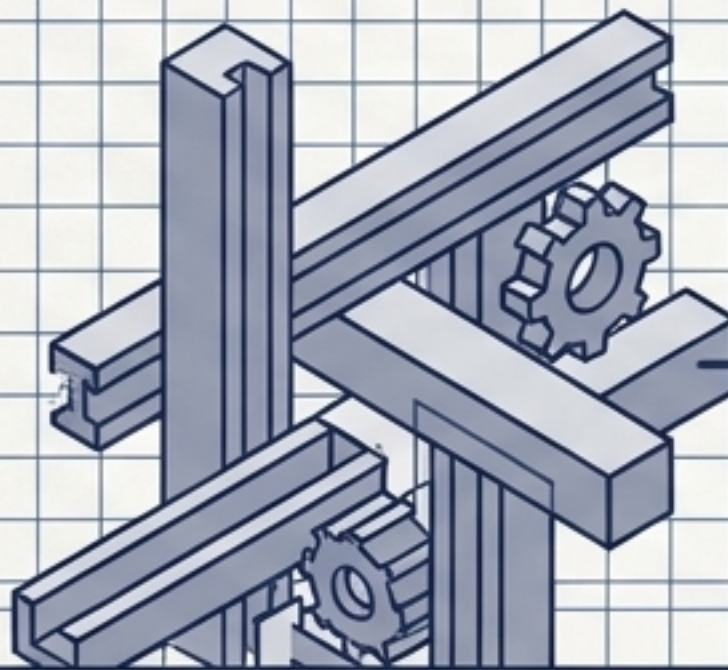
Core Insight: TDD is not only a testing practice but also a design practice.

PROJECT: SOFTWARE ARCHITECTURE DIAGNOSTICS			
DRAWN BY: AI ARCHITECT DATE:	SHEET NO:	REV:	
2024-10-27	003	A	

Trusting AI is Failing

Classical Code: Deterministic

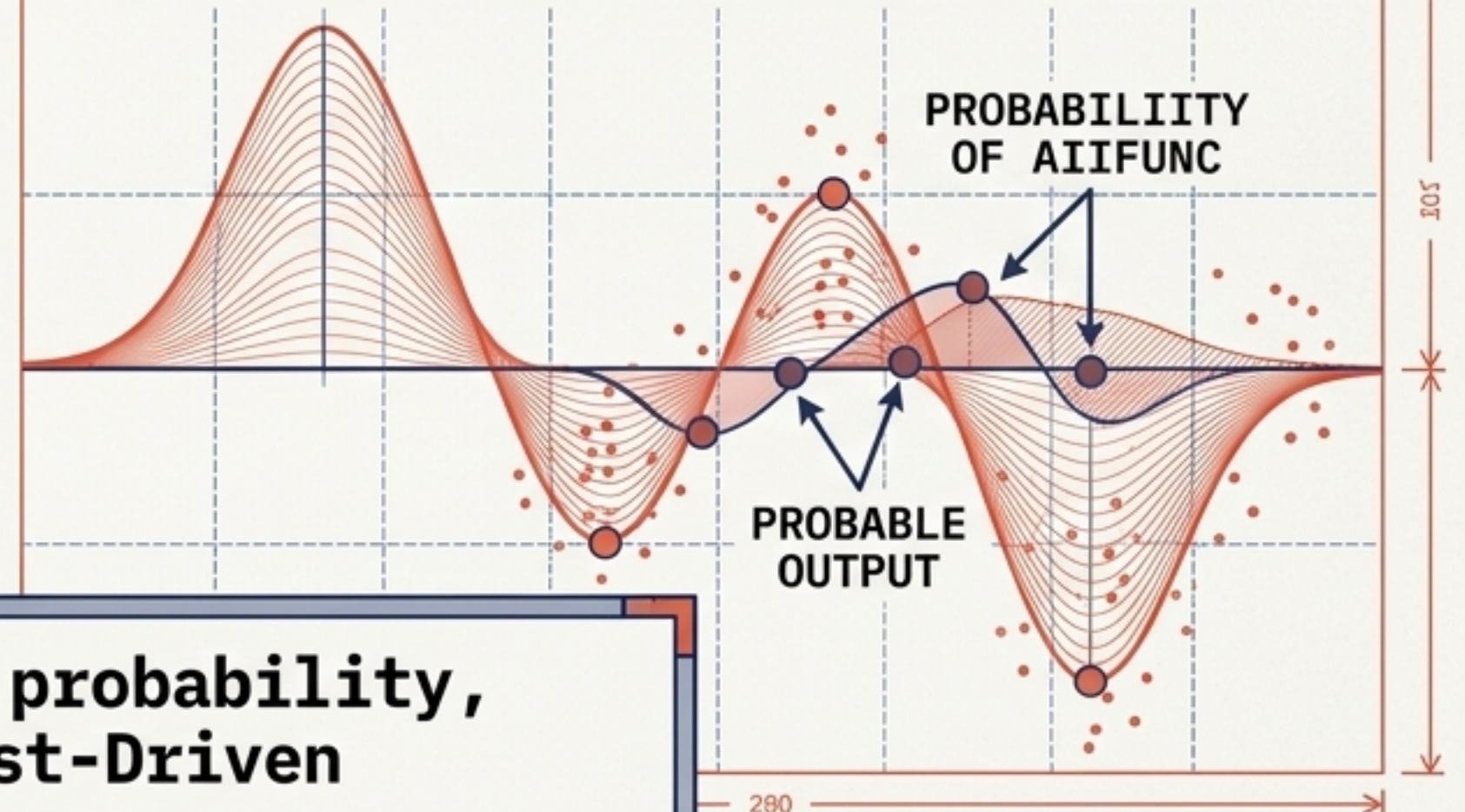
Logic written by humans executes exactly as commanded. A formal proof.



EXACT
EXECUTION

AI Code: Statistical

LLMs predict the next likely token. Passing a test represents a statistical probability of correctness, not a formal proof.



The Danger: Because AI introduces probability, the rigorous rules of classical Test-Driven Development are more critical than ever.

PROJECT: SOFTWARE ARCHITECTURE DIAGNOSTICS

DRAWN BY: AI ARCHITECT

2024-10-27

SHEET NO: REV: A

004 A

The Confirmation Bias Trap

Step 1: Prompt AI for a solution

(e.g., Write a PII redaction function)

Step 2: AI outputs plausible-looking code

Step 3 (The Trap): Confirmation Bias

Developer reads code, nods along, and writes a weak test designed to pass the AI's logic rather than testing the requirement.

HAZARD

HAZARD

▲ FAILURE STATE

FALSE POSITIVE ▲

Result: False Green

System passes the test, but edge cases remain unprotected.

PROJECT: SOFTWARE ARCHITECTURE DIAGNOSTICS

DRAWN BY: AI ARCHITECT

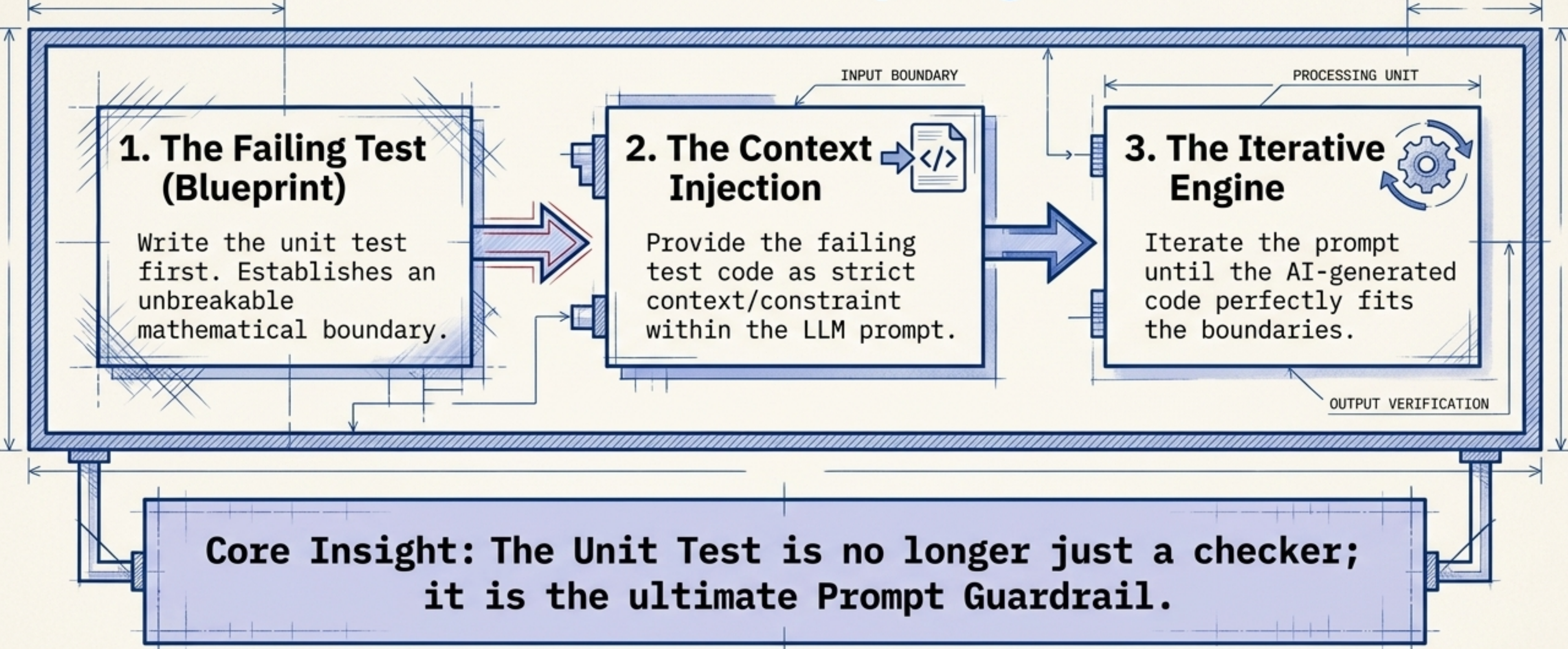
2024-10-27

SHEET NO: REV: A

005

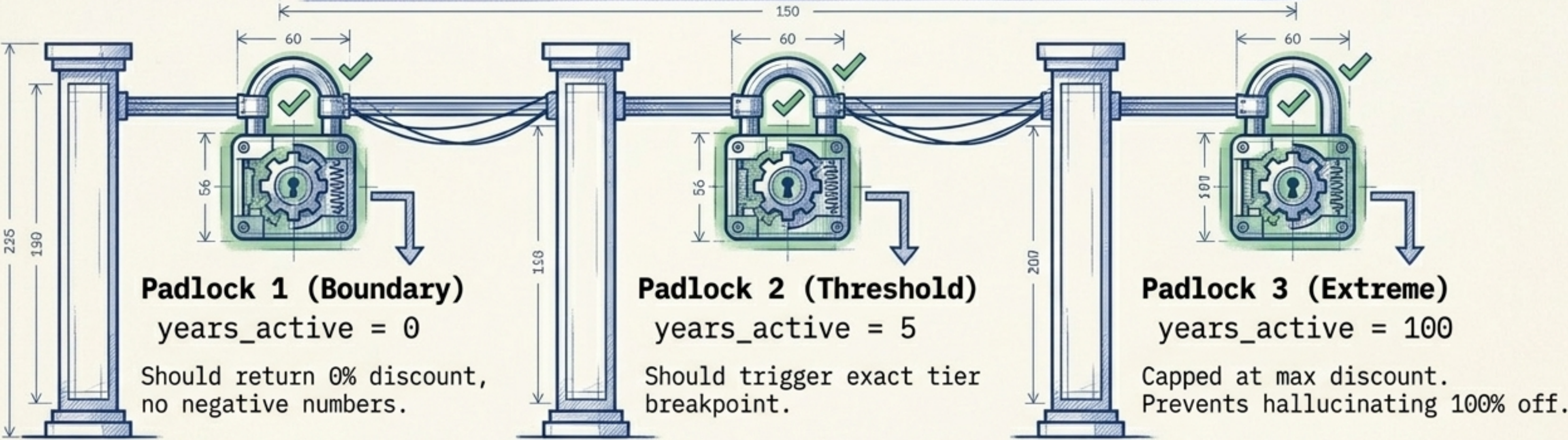
A

Test-Driven Prompting (TDP)



Building Guardrails: The Edge Case Cage

PROMPT: Write a function that calculates a loyalty discount based on a 'years active' integer.



By defining extremes first, the AI is mathematically blocked from hallucinating logic.

	PROJECT: SOFTWARE ARCHITECTURE DIAGNOSTICS			
	DRANN BY: AI ARCHITECT			
	DATE: 2024-10-27	SHICET NO: 007		REV: A

The Evolution of Trust

Waterfall

Agile / TDD

Test-Driven Prompting

Code → Test

Test → Code

Test → Prompt → Auto-Generate

High risk of bugs,
high cost of rework.

High trust, but slow human
execution speed.

High trust + High speed.

AI hasn't killed the unit test. It has made the unit test the most important artifact in software engineering.